Machine Learning Zoomcamp

Module 3 — Classification: Churn Prediction

Study notes with equations and charts

Based on the DataTalksClub course notes, `03-classification`

Condensed and re-organised from
https://github.com/DataTalksClub/machine-learning-zoomcamp/tree/main/03-classification.
All charts are computed on the real Telco churn dataset (7 043 customers, `random_state=1`). Numbers can differ from the videos in the 3rd decimal depending on library versions.

Contents

1 The Churn Problem **2**

2 Data Preparation **2**

3 Validation Framework with Scikit-Learn **2**

4 EDA and the Churn Rate **3**

5 Feature Importance **3**
5.1 Churn rate per group, difference and risk ratio . . . 3
5.2 Mutual information (categorical features) . . . 3
5.3 Correlation (numerical features) . . . 4

6 One-Hot Encoding with `DictVectorizer` **4**

7 Logistic Regression **5**

8 Training with Scikit-Learn and Accuracy **5**

9 Model Interpretation **6**
9.1 A small model you can compute by hand . . . 7
9.2 Weights as odds ratios . . . 8

10 Using the Model **8**

11 Summary **8**

A Appendix: How Logistic Regression Is Trained **8**
A.1 Likelihood and log loss . . . 9
A.2 Gradient and why it must be iterative . . . 9
A.3 Regularization is on by default . . . 9

1 The Churn Problem

Churn = a customer stops using a service. A telecom company wants to spot customers about to leave and offer them a discount *before* they go. For every customer we want a **score** in $[0, 1]$: the probability of churning.

This is **binary classification**:

$$g(\mathbf{x}_i) \approx y_i, \qquad y_i \in \{0, 1\}, \qquad y_i = \begin{cases} 1 & \text{churned (positive class)} \\ 0 & \text{stayed (negative class)} \end{cases}$$

g is learned on last month's customers (whose outcome we know) and applied to current ones. Features: demographics (gender, senior citizen, partner, dependents), services (phone, internet, security, backup, streaming, ...), account (contract, billing, payment method), charges (monthly, total) and tenure (months with the company).

2 Data Preparation

```
df.columns = df.columns.str.lower().str.replace(' ', '_')
for c in categorical_columns: # string columns
    df[c] = df[c].str.lower().str.replace(' ', '_')
df.totalcharges = pd.to_numeric(df.totalcharges, errors='coerce').fillna(0)
df.churn = (df.churn == 'yes').astype(int) # target -> 0/1
```

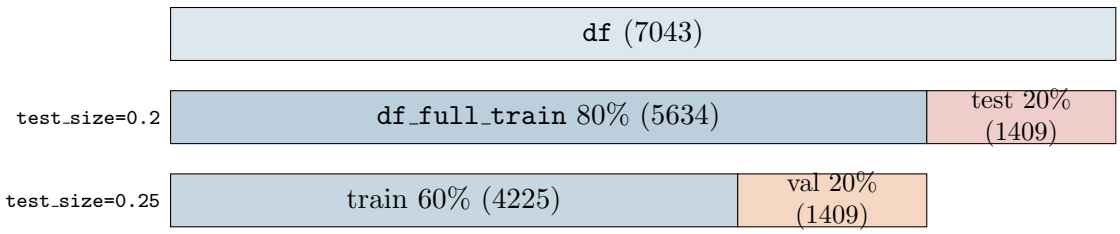
- `totalcharges` was read as text: missing values were a space, which became `"_"` after normalisation. `errors='coerce'` turns unparseable strings into NaN (11 rows). They are brand-new customers who have not been billed yet, so 0 is a sensible fill.
- `seniorcitizen` is stored as 0/1 but is really categorical.

pandas 3.x

As in Module 2: select string columns with `df.select_dtypes(include=['object', 'string'])`, because `df.dtypes == 'object'` finds nothing in pandas 3.

3 Validation Framework with Scikit-Learn

`train_test_split` splits in two, so we call it twice:



Why 0.25 the second time? Validation must be 20% of the *original* data, and it is taken from the remaining 80%: $0.25 \times 0.8 = 0.2$.

```
df_full_train, df_test = train_test_split(df, test_size=0.2, random_state=1)
df_train, df_val = train_test_split(df_full_train, test_size=0.25, random_state=1)
y_train = df_train.churn.values; del df_train['churn'] # same for val, test
```

4 EDA and the Churn Rate

No missing values remain. The **churn rate** is the fraction of positives, and for a 0/1 column it is simply the mean:

$$\text{churn rate} = \frac{1}{m}\sum_{i=1}^{m} y_i = \bar{y} = 0.270.$$

So 27% churned and 73% stayed: the classes are *imbalanced*. Three variables are numerical (`tenure`, `monthlycharges`, `totalcharges`); 16 are categorical (2–4 values each, `paymentmethod` has 4).

5 Feature Importance

5.1 Churn rate per group, difference and risk ratio

For a group G (e.g. customers with `partner = no`):

$$\text{diff}_G = \bar{y}_G - \bar{y}, \qquad \text{risk}_G = \frac{\bar{y}_G}{\bar{y}} \quad \begin{cases} > 1 & \text{churns more than average} \\ = 1 & \text{average} \\ < 1 & \text{churns less} \end{cases}$$

Example: no partner $\bar{y}_G = 0.330 \Rightarrow$ risk $= 0.330/0.270 = 1.22$ (22% more churn); with partner $0.205 \Rightarrow 0.76$.

```
for c in categorical:
    df_group = df_full_train.groupby(c).churn.agg(['mean', 'count'])
    df_group['diff'] = df_group['mean'] - global_churn
    df_group['risk'] = df_group['mean'] / global_churn
    display(df_group)
```

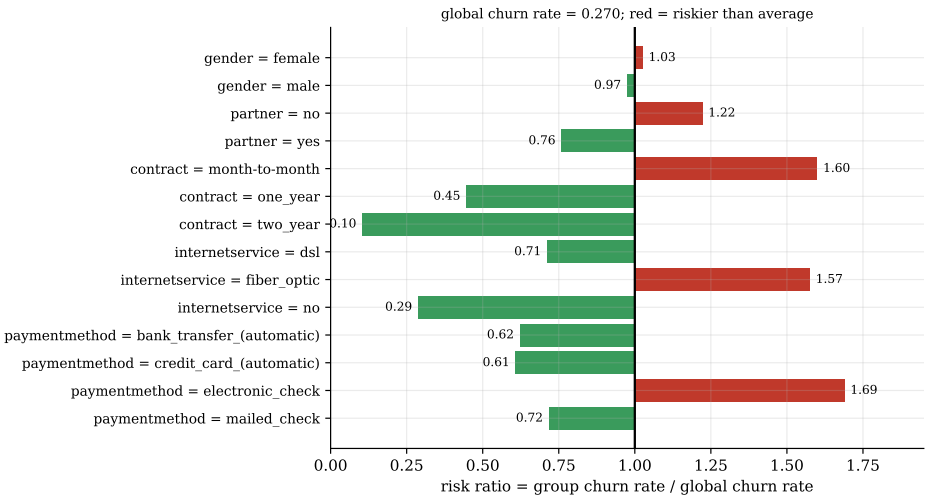


Figure 1: Risk ratios for five variables. Gender is flat (useless); month-to-month contracts, fibre optic and electronic check are the riskiest groups, two-year contracts the safest.

5.2 Mutual information (categorical features)

Risk ratios compare groups *inside* a variable. To compare *variables*, we use the mutual information between a feature X and the target Y: how much knowing X reduces our uncertainty about Y.

$$I(X;Y) = \sum_x \sum_y p(x,y)\,\log\frac{p(x,y)}{p(x)\,p(y)} \quad \geq 0,$$

with $I = 0$ exactly when X and Y are independent ($p(x, y) = p(x)p(y)$). It is symmetric in X and Y.

By hand, for `contract` (joint frequencies on the full train set):

$p(x, y)$	$y = 0$	$y = 1$	$p(x)$
month-to-month	0.313	0.238	0.551
one_year	0.185	0.025	0.211
two_year	0.232	0.007	0.239
$p(y)$	0.730	0.270	1

E.g. the cell (month-to-month, churn) contributes $0.238 \log \frac{0.238}{0.551 \cdot 0.270} = +0.112$: churn is much more frequent there than independence would predict. Summing all six cells gives $I \approx 0.098$ nats, the value of `mutual_info_score`.

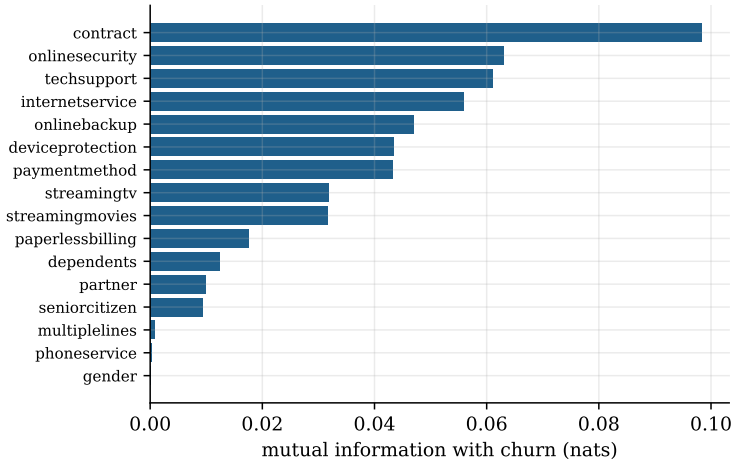


Figure 2: Mutual information of every categorical variable with churn: contract first, gender last.

5.3 Correlation (numerical features)

Pearson correlation between a feature x and the target y:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}} \in [-1, 1].$$

Sign = direction, $|r|$ = strength (rule of thumb: < 0.2 low, 0.2–0.5 medium, > 0.5 strong). Results: tenure -0.35, monthly charges $+0.20$, total charges -0.20.

6 One-Hot Encoding with `DictVectorizer`

Each categorical variable with K values becomes K binary columns, exactly one of which is "hot":

$$x_{i,\, c=v} = \mathbf{1}[c_i = v].$$

gender	contract	g=F	g=M	c=M2M	c=1Y	c=2Y
F	2Y	1	0	0	0	1
M	M2M	0	1	1	0	0
M	1Y	0	1	0	1	0

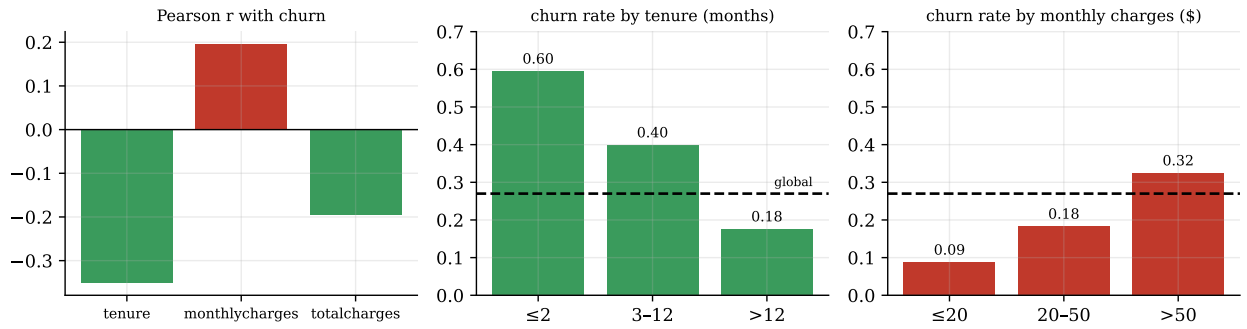


Figure 3: Correlations confirmed by binning: new customers ($\leq$2 months) churn at 60%, loyal ones (>12) at 18%; customers paying over \$50/month churn at 32%.

`DictVectorizer` takes one dictionary per row; string values are one-hot encoded, numbers pass through unchanged.

```
dv = DictVectorizer(sparse=False)
train_dict = df_train[categorical + numerical].to_dict(orient='records')
X_train = dv.fit_transform(train_dict) # learns categories: 4225 x 45
X_val = dv.transform(val_dict) # transform only!
dv.get_feature_names_out() # 'contract=one_year', ..., 'tenure'
```

fit on train only

Calling `fit` again on validation could reorder columns or add categories that only appear there, so the matrix would no longer match the trained model.

45 columns = 42 one-hot columns (sum of categories over 16 variables) + 3 numerical.

7 Logistic Regression

Linear regression outputs any real number; a probability must lie in $[0, 1]$. Logistic regression keeps the linear **score** and squashes it with the **sigmoid**:

$$z_i = w_0 + \mathbf{x}_i^T \mathbf{w}, \qquad \boxed{g(\mathbf{x}_i) = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}}$$

Properties worth remembering: $\sigma(0) = \frac{1}{2}$, $\sigma(-z) = 1 - \sigma(z)$, and $z > 0 \Leftrightarrow g > 0.5$. It is still a *linear model*: the decision boundary $z = 0$ is a hyperplane in feature space.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
def logistic_regression(xi):
    score = w0 + sum(xi[j] * w[j] for j in range(len(w)))
    return sigmoid(score) # the only difference from linear regression
```

8 Training with Scikit-Learn and Accuracy

```
model = LogisticRegression(solver='lbfgs')
model.fit(X_train, y_train)
model.intercept_[0], model.coef_[0] # w0 and the 45 weights
y_pred = model.predict_proba(X_val)[:, 1] # soft: P(churn); column 1 = class 1
```

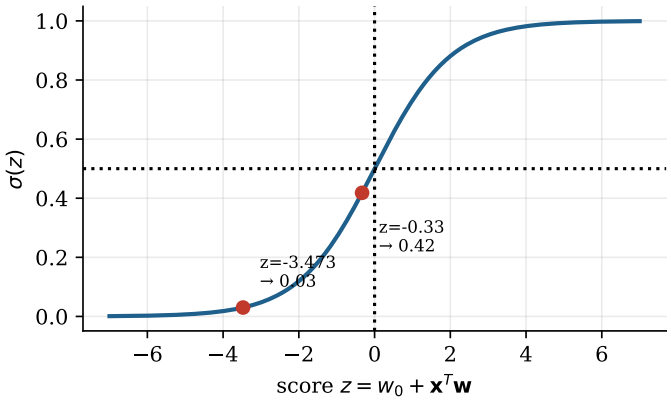


Figure 4: The sigmoid: $\sigma(0) = 0.5$, large positive scores go to 1, large negative ones to 0. The two points are the hand-scored customers of §9.

```
churn_decision = (y_pred >= 0.5) # hard: 0/1 with our own threshold
(y_val == churn_decision).mean() # accuracy ~ 0.80
```

`predict` gives hard 0/1 labels (threshold 0.5 built in); `predict_proba` gives probabilities (two columns summing to 1), which lets us choose the threshold ourselves.

$$\text{accuracy} = \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\big[\,[g(\mathbf{x}_i) \geq t] = y_i\,\big].$$

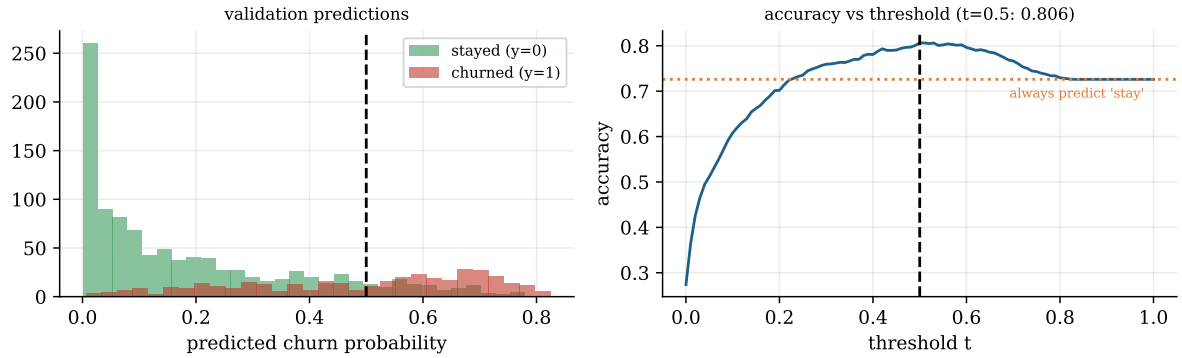


Figure 5: Left: predicted probabilities on validation, by actual outcome; most stayers get low scores, churners are spread out. Right: accuracy as a function of the threshold. A dummy model that always predicts "stay" already gets 73% (dotted line), so 80% is good but less impressive than it sounds — Module 4 covers better metrics.

Convergence warning

On unscaled features (`totalcharges` reaches thousands) `lbfgs` may stop before converging. Fixes: `max_iter=1000`, or scale numerical features with `StandardScaler` (see `notebook-scaling-ohe.ipynb`).

9 Model Interpretation

Pair weights with names: `dict(zip(dv.get_feature_names_out(), model.coef_[0].round(3)))`. As in linear regression, a positive weight pushes the score (and the probability) up.

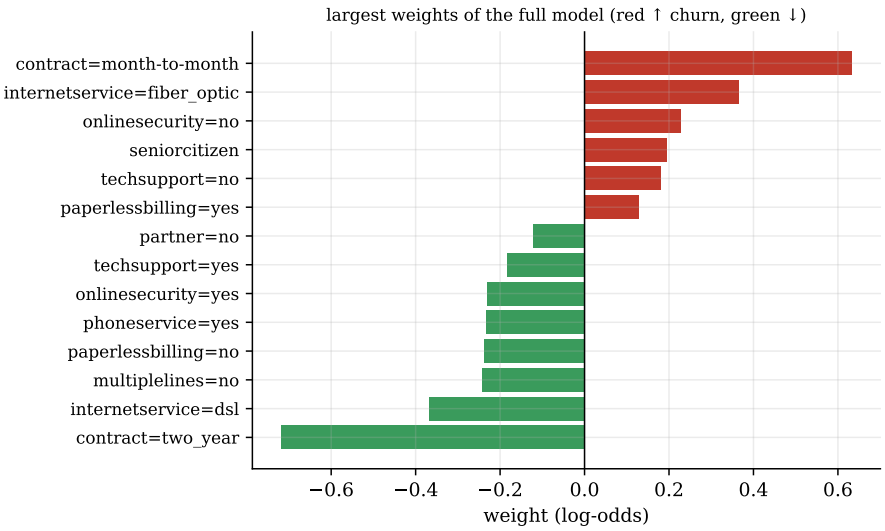


Figure 6: Largest weights of the full 45-feature model. They agree with the risk ratios: month-to-month and fibre optic increase churn, two-year contracts decrease it. With one-hot encoding, a customer "uses" only the weight of their own category.

9.1 A small model you can compute by hand

Using only `contract, tenure, monthlycharges` (5 columns):

$$w_0 = -2.47; \quad w_{\text{m2m}} = 0.97,\ w_{\text{1y}} = -0.025,\ w_{\text{2y}} = -0.949; \quad w_{\text{charges}} = 0.027,\ w_{\text{tenure}} = -0.036.$$

Customer A (two-year contract, \$30/month, 24 months):

$$z = -2.47 - 0.949 + 30 \cdot 0.027 - 24 \cdot 0.036 = -3.473, \qquad \sigma(-3.473) = 0.030 \ \ (3\%).$$

Customer B (month-to-month, \$50/month, 5 months):

$$z = -2.47 + 0.97 + 50 \cdot 0.027 - 5 \cdot 0.036 = -0.33, \qquad \sigma(-0.33) = 0.42 \ \ (42\%).$$

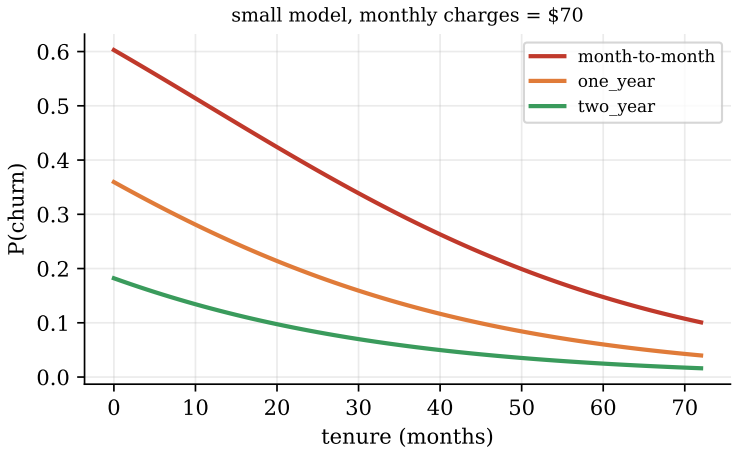


Figure 7: The small model's churn probability vs tenure (at \$70/month) for each contract type. Each curve is a stretch of the same sigmoid, shifted by its contract weight: month-to-month starts near 60%, two-year below 20%.

9.2 Weights as odds ratios

The score is the **log-odds**: since $p = \sigma(z)$,

$$z = \log \frac{p}{1-p} \quad \Longrightarrow \quad \frac{p}{1-p} = e^{w_0}\, e^{w_1 x_1} \cdots e^{w_n x_n}.$$

Increasing x_j by one unit multiplies the odds of churn by e^{w_j}:

- month-to-month vs two-year: $e^{0.97-(-0.949)} = e^{1.92} \approx 6.8\times$ the odds;
- +\$10/month: $e^{0.27} \approx 1.31$ (+31% odds); +12 months tenure: $e^{-0.43} \approx 0.65$ (−35% odds).

10 Using the Model

Retrain on train + validation (new `DictVectorizer` too: encoder and model always go together), check on test:

```
dicts_full_train = df_full_train[categorical + numerical].to_dict(orient='records')
dv = DictVectorizer(sparse=False); X_full_train = dv.fit_transform(dicts_full_train)
model = LogisticRegression(solver='lbfgs').fit(X_full_train, df_full_train.churn.values)
y_pred = model.predict_proba(dv.transform(dicts_test))[:, 1]
((y_pred >= 0.5) == y_test).mean() # ~0.81 test vs ~0.80 val
customer = dicts_test[-1] # one dict = one request
model.predict_proba(dv.transform([customer]))[0, 1] # ~0.60 -> send a discount
```

Test accuracy ≈ validation accuracy, so no sign of overfitting. The last test customer (month-to-month, fibre optic, electronic check, 17 months) gets ≈60% and did in fact churn. To deploy, both the vectorizer and the model must be saved (Module 5).

11 Summary

Problem	Binary classification, $y \in \{0, 1\}$; the model outputs $P(y = 1)$.
Preparation	Fix `totalcharges` (`to_numeric`, `coerce`, fill 0); target → 0/1.
Validation	`train_test_split` twice (0.2 then 0.25) → 60/20/20.
Churn rate	$\bar{y} = 0.27$; group rates give **diff** $= \bar{y}_G - \bar{y}$ and **risk** $= \bar{y}_G / \bar{y}$.
Importance	Mutual information for categorical (contract 0.098), Pearson r for numerical (tenure −0.35).
Encoding	One-hot with `DictVectorizer`: `fit` on train, `transform` elsewhere; 45 columns.
Model	$g(\mathbf{x}) = \sigma(w_0 + \mathbf{x}^T \mathbf{w})$; weights are log-odds, e^{w_j} is an odds ratio.
Evaluation	Accuracy ≈ 0.80 (val), 0.81 (test); baseline "always stay" = 0.73.

Explore more: drop the least useful features; redo Module 2 with `train_test_split`, `LinearRegression` and `Ridge`; try `OneHotEncoder` and `StandardScaler`; practise on lead scoring or credit default data.

A Appendix: How Logistic Regression Is Trained

In Module 2 the weights came from a formula (the normal equation). Logistic regression has *no* closed-form solution; the weights are found by iterative optimisation (`lbfgs`). This appendix explains what is being optimised.

A.1 Likelihood and log loss

For one customer the model says $p_i = \sigma(z_i) = P(y_i = 1)$. The probability it assigns to what actually happened is p_i if $y_i = 1$ and $1 - p_i$ if $y_i = 0$, compactly $p_i^{y_i}(1 - p_i)^{1-y_i}$. Training picks the $\mathbf{w}$ that makes the observed data most likely; taking minus the log turns the product into a sum, the **log loss** (binary cross-entropy):

$$\boxed{L(\mathbf{w}) = -\sum_{i=1}^{m} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big]}$$

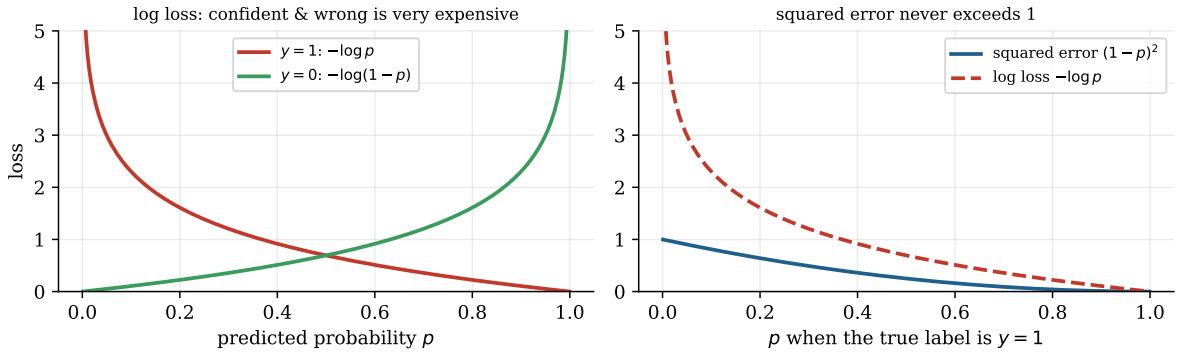


Figure 8: Left: the loss of one example. Predicting $p = 0.01$ for a churner costs $-\log 0.01 = 4.6$; predicting 0.99 costs 0.01. Right: unlike squared error, log loss punishes confident mistakes without bound.

A.2 Gradient and why it must be iterative

Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, the gradient takes a remarkably simple form:

$$\nabla L = \mathbf{X}^T(\mathbf{p} - \mathbf{y}), \qquad \mathbf{p} = \sigma(\mathbf{X}\mathbf{w}).$$

Compare with linear regression, $\nabla = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$: same "features × errors" structure. But here $\mathbf{p}$ depends on $\mathbf{w}$ through the non-linear sigmoid, so $\mathbf{X}^T(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y}) = \mathbf{0}$ cannot be solved in closed form. Instead the solver starts from some $\mathbf{w}$ and repeatedly steps downhill, e.g. gradient descent $\mathbf{w} \leftarrow \mathbf{w} - \eta\, \mathbf{X}^T(\mathbf{p} - \mathbf{y})$; `lbfgs` is a smarter variant that also uses curvature. L is convex (one bowl, no local minima), so the solver reaches the global optimum — provided it runs long enough, hence the convergence warning and the value of feature scaling.

A.3 Regularization is on by default

Scikit-Learn actually minimises

$$\tfrac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{m} \text{logloss}_i, \qquad C = 1 \text{ by default},$$

the same L2 (ridge) idea as Module 2, with $C = 1/r$: smaller C means stronger regularization and smaller weights. C is a hyperparameter to tune on the validation set.